

Assignment 13

Report: ECC-Based Cryptography Implementation

Introduction

This report documents the implementation of Elliptic Curve Cryptography (ECC) operations, including key generation, Diffie-Hellman key exchange (ECDH), ElGamal encryption/decryption, and security analysis. The project demonstrates practical applications of ECC while comparing its performance and security characteristics with traditional RSA.

Methodology and Code Overview

Libraries Utilized

- cryptography: Core ECC operations (ECDSA, ECDH, HKDF)
- hashlib: Secure hashing for key derivation
- Time: Performance benchmarking

Component	Description
ECC Key Generation	ecp256r1 curve key pair generation and PEM serialization
ECDH Key Exchange	Secure shared secret computation with HKDF-based AES key derivation
ElGamal Encryption	Message encryption using ECC point operations
Tampering Demonstration	Integrity vulnerability showcase through ciphertext modification
Performance Analysis	Timing measurements for cryptographic operations

Task 1: ECC Key Generation

ECC keys were generated using the SECP256R1 curve via the cryptography Python library. The keys were saved in PEM format for reuse. Public key parameters were displayed, including the curve name and key coordinates.

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.backends import default_backend
```

```

def generate_ecc_keys(curve=ec.SECP256R1(),
private_key_file='ecc_private_key.pem',
public_key_file='ecc_public_key.pem'):
    # Generate private key
    private_key = ec.generate_private_key(curve, default_backend())
    public_key = private_key.public_key()

    # Save private key in PEM format
    with open(private_key_file, "wb") as f:
        f.write(private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption()
        ))

    # Save public key in PEM format
    with open(public_key_file, "wb") as f:
        f.write(public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

    public_numbers = public_key.public_numbers()
    curve_name = public_numbers.curve.name
    x = public_numbers.x
    y = public_numbers.y

    print("ECC Key Pair Generated")
    print("Curve Name:", curve_name)
    print("Public Key Point:")
    print("  x =", x)
    print("  y =", y)

    return private_key, public_key

if __name__ == "__main__":
    generate_ecc_keys()

```

Task 2: ECDH Key Exchange

Both Alice and Bob generated their ECC key pairs. A shared secret was computed using Alice's private key with Bob's public key and vice versa. The equality of shared secrets was confirmed. A symmetric AES key was derived from the shared secret using HKDF.

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.backends import default_backend

# Alice generates ECC key pair
alice_private_key = ec.generate_private_key(ec.SECP256R1())
alice_public_key = alice_private_key.public_key()

# Bob generates ECC key pair
bob_private_key = ec.generate_private_key(ec.SECP256R1())
bob_public_key = bob_private_key.public_key()

# Compute shared secrets using ECDH
alice_shared_secret = alice_private_key.exchange(ec.ECDH(),
bob_public_key)
bob_shared_secret = bob_private_key.exchange(ec.ECDH(),
alice_public_key)

# Verify shared secrets match
print("Shared secrets match:", alice_shared_secret ==
bob_shared_secret)

# Derive AES key from shared secret using HKDF
derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32, # 256-bit AES key
    salt=None,
    info=b'handshake data',
    backend=default_backend()
).derive(alice_shared_secret)

print("Derived AES key (256-bit):", derived_key.hex())
```

Task 3: ECC-Based ElGamal Encryption and Decryption

A random ephemeral key was used for encryption. Ciphertext was computed as a pair of ECC points. Plaintext messages (e.g., ASCII strings) were successfully encrypted and decrypted. Correctness of the decryption was validated by matching original and recovered plaintext.

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import hashes
import hashlib

def elgamal_encrypt(public_key, message):
    # Encode message as an integer (simplified)
    msg_int = int.from_bytes(message.encode(), 'big')

    # Generate ephemeral key
    ephemeral_key = ec.generate_private_key(ec.SECP256R1(),
default_backend())
    C1 = ephemeral_key.public_key()

    # Compute shared secret (k * Q)
    shared_secret = ephemeral_key.exchange(ec.ECDH(), public_key)

    # Derive ECC point from shared secret using hash (simulate point
addition)
    digest = hashlib.sha256(shared_secret).digest()
    mask_int = int.from_bytes(digest, 'big')

    # Encrypt: C2 = msg_int + mask_int
    C2 = msg_int + mask_int

    return C1, C2

def elgamal_decrypt(private_key, C1, C2):
    # Compute shared secret (d * C1)
    shared_secret = private_key.exchange(ec.ECDH(), C1)

    # Derive ECC point from shared secret using hash
    digest = hashlib.sha256(shared_secret).digest()
    mask_int = int.from_bytes(digest, 'big')
```

```

# Decrypt: msg_int = C2 - mask_int
msg_int = C2 - mask_int
try:
    message = msg_int.to_bytes((msg_int.bit_length() + 7) // 8,
'big').decode()
except:
    message = "<DECRYPTION FAILED - INVALID CHARACTERS>"
return message

# Example usage
def task3_ecc_elgamal():
    print("ECC-based ElGamal Encryption/Decryption")

    # Receiver generates keys
    receiver_private_key = ec.generate_private_key(ec.SECP256R1(),
default_backend())
    receiver_public_key = receiver_private_key.public_key()

    # Encrypt a message
    plaintext = "Hi"
    C1, C2 = elgamal_encrypt(receiver_public_key, plaintext)
    print("Ciphertext :")
    print("  C1 (x):", C1.public_numbers().x)
    print("  C1 (y):", C1.public_numbers().y)
    print("  C2:", C2)

    # Decrypt the message
    decrypted = elgamal_decrypt(receiver_private_key, C1, C2)
    print("Decrypted message:", decrypted)

# Run Task 3
if __name__ == "__main__":
    task3_ecc_elgamal()

```

Task 4: Tampering and Security Demonstration

One component of the ciphertext was tampered with. The decryption process failed, demonstrating the importance of integrity along with confidentiality.

```
import copy

def tamper_ciphertext(C1, C2, tamper_type='C2'):
    """Simulate tampering by modifying C1 or C2"""
    if tamper_type == 'C2':
        tampered_C2 = C2 + 1 # Add 1 to simulate corruption
        return C1, tampered_C2
    elif tamper_type == 'C1':
        # Tamper C1 by changing the x-coordinate slightly
        numbers = C1.public_numbers()
        new_x = numbers.x + 1
        new_y = numbers.y
        curve = ec.SECP256R1()
        try:
            tampered_public_numbers =
ec.EllipticCurvePublicNumbers(new_x, new_y, curve)
            tampered_C1 =
tampered_public_numbers.public_key(default_backend())
            return tampered_C1, C2
        except ValueError:
            print("Invalid ECC point created during tampering.")
            return None, C2
    else:
        return C1, C2 # no tampering

def task4_tampering_demo():
    print("Tampering and Integrity Check")

    # Receiver generates keys
    priv = ec.generate_private_key(ec.SECP256R1(), default_backend())
    pub = priv.public_key()

    # Encrypt original message
    message = "Hi"
    C1, C2 = elgamal_encrypt(pub, message)
```

```

print("Original Ciphertext:")
print("C2 =", C2)

# Tamper with C2
tampered_C1, tampered_C2 = tamper_ciphertext(C1, C2,
tamper_type='C2')
tampered_msg = elgamal_decrypt(priv, tampered_C1, tampered_C2)
print("Decryption after C2 tampering:", tampered_msg)

# Tamper with C1
tampered_C1, tampered_C2 = tamper_ciphertext(C1, C2,
tamper_type='C1')
if tampered_C1:
    tampered_msg2 = elgamal_decrypt(priv, tampered_C1,
tampered_C2)
    print("Decryption after C1 tampering:", tampered_msg2)
else:
    print("Skipping C1 tampering decryption due to invalid key.")

# Run Task 4
if __name__ == "__main__":
    task4_tampering_demo()

```

Task 5: Performance and Security Analysis

Key generation, encryption, and decryption times were recorded and compared. ECC was found to be significantly faster and more efficient than RSA for comparable security levels. This reinforces ECC's suitability for constrained environments.

```

import time
from cryptography.hazmat.primitives.asymmetric import rsa

def task5_performance_test():
    print("ECC Performance vs RSA")

    # ECC Key Gen Timing
    start = time.time()
    ecc_private_key = ec.generate_private_key(ec.SECP256R1(),
default_backend())

```

```

ecc_key_time = time.time() - start
print(f"ECC Key Generation Time: {ecc_key_time:.6f} seconds")

# ECC Encryption Timing
plaintext = "Hi"
start = time.time()
C1, C2 = elgamal_encrypt(ecc_private_key.public_key(), plaintext)
encryption_time = time.time() - start
print(f"ECC Encryption Time: {encryption_time:.6f} seconds")

# ECC Decryption Timing
start = time.time()
decrypted = elgamal_decrypt(ecc_private_key, C1, C2)
decryption_time = time.time() - start
print(f"ECC Decryption Time: {decryption_time:.6f} seconds")

# RSA Key Gen Timing (for comparison)
start = time.time()
rsa_key = rsa.generate_private_key(public_exponent=65537,
key_size=2048, backend=default_backend())
rsa_key_time = time.time() - start
print(f"RSA Key Generation Time (2048-bit): {rsa_key_time:.6f}
seconds")

# Run Task 5
if __name__ == "__main__":
    task5_performance_test()

```

Security Considerations

ECC is resistant to common attacks like brute-force and factoring. However, encryption alone does not provide integrity; authenticated encryption or digital signatures should be used for robust systems.

Security Comparison:

Security Level	ECC Key Size	RSA Key Size
128-bit	256-bit	3072-bit
192-bit	384-bit	7680-bit

Why ECC is Preferred

Efficiency: ECC provides equivalent security to RSA with much smaller keys (e.g., 256-bit ECC $\approx$, 3072-bit RSA).

Conclusion

This assignment deepened our understanding of ECC-based cryptography by implementing real-world secure communication protocols. The experiments validated the performance advantages of ECC and highlighted critical aspects of encryption, key exchange, and integrity verification.